

Škálovatelné zpracování telemetrie zařízení

Cílem úkolu je ukázat architektonické uvažování nad distribuovaným systémem, zejména práci s vysokým průtokem dat, škálováním, duplicitami, pořadím událostí a race conditions.

Použití AI nástrojů je povoleno. Při následné diskuzi ale očekáváme, že budete schopni vysvětlit svůj návrh, klíčová rozhodnutí, kompromisy i implementaci.

Kontext

Máme větší množství zařízení, která přes dlouhodobé socketové spojení posílají telemetrická data.

Telemetrie může obsahovat například (tématicky):

- aktuální provozní stav zařízení,
- naměřené hodnoty, například teplotu, cpu, ram
- čítače, například počet provedených operací nebo celkový čas jak dlouho bylo zařízení zapnuto,
- diagnostické či chybové informace.

Zadání

Navrhněte a implementujte systém obsahující:

1. Emulátor zařízení

- Umí spustit konfigurovatelný počet zařízení.
- Zařízení se připojují přes socket k ingest službě.
- Generují události.

2. Socket ingest službu

- Přijímá zprávy od zařízení.
- Validuje je a předává je do RabbitMQ.
- Musí být možné ji horizontálně škálovat.

3. Processing service

- Konzumuje zprávy z RabbitMQ.
- Ukládá zpracované eventy do MongoDB.

- Vytváří a udržuje aktuální stav každého zařízení.
- Musí být možné ji horizontálně škálovat.

4. Docker Compose (vývojový, nikoliv produkční)

- Musí umožnit spustit celý systém včetně RabbitMQ, MongoDB a aplikací.
- Mělo by být možné snadno navýšit počet emulovaných zařízení i instancí jednotlivých služeb.
- Projekt vytvořte jako pnpm monorepo v přísně typovaném TypeScriptu pro Node.js. Konkrétní knihovny a frameworky jsou na vás.

Technické požadavky

Návrh musí počítat s tím, že výsledný stav zařízení musí odpovídat logickému pořadí unikátních událostí, které zařízení vytvořilo.

Například starší telemetrie nesmí přepsat novější známý stav zařízení. Duplicitní zpráva nesmí způsobit duplicitní business efekt — například opakované započítání změny čítače, opakované vytvoření alertu nebo nekonzistentní stav zařízení.

Neomezujeme vás konkrétním datovým modelem ani strategií řešení. Při návrhu se ale zamyslete zejména nad tím:

- jaká metadata musí telemetrická zpráva obsahovat, aby bylo možné řešit výše popsané problémy konzistentně (ošetření race conditions tak, aby to mělo co nejmenší dopad na propustnost systému)
- co znamená „aktuální stav“ zařízení a podle čeho se určuje jeho novost
- které operace musí být atomické a jak je atomicita zaručena
- jak umožnit paralelní zpracování různých zařízení, aniž by došlo ke konfliktům uvnitř jednoho zařízení

Testy

Součástí řešení musí být automatizované testy

Preferujeme, aby alespoň část testů byla integračních, nad skutečnými instancemi MongoDB a RabbitMQ. Rozsah řešení zvolte přiměřeně času.

Dokumentace

Do README.md uveďte:

- stručný popis architektury a toku dat
- návod na spuštění systému a testů
- konfiguraci emulátoru
- jak spustit více instancí ingest a processing služby
- známé limity a vědomé kompromisy
- co byste při více času doplnili nebo řešili jinak.
- Krátký diagram architektury je vítán.

Odevzdání

Projekt odevzdejte jako repository na GitHubu nebo GitLabu.

Repository musí obsahovat zdrojové kódy, Docker Compose konfiguraci, dokumentaci a testy.

Očekáváme běžnou a smysluplnou commit historii odpovídající tomu, jak standardně odevzdáváte práci v týmu.

Po odevzdání proběhne technická diskuze nad:

- architekturou a datovým modelem
- zvolenými metadaty telemetrických zpráv
- řešením race conditions
- škálováním a odolností při selhání
- testovací strategií
- kompromisy a možnými alternativami.